

GO: MODULI, TOOL



PACKAGE



PACKAGE

- Un programma Go è una composizione di **pacchetti**.
- Il pacchetto principale è `main`.
- La funzione iniziale (punto d'ingresso) è `main()` all'interno del pacchetto `main`.
- I pacchetti vengono importati utilizzando il loro **percorso**, ad esempio `encoding/json` o `fmt`.
- Ogni pacchetto deve risiedere in una **directory dedicata**.
- Un **modulo** è un gruppo di pacchetti.



MODULE

- Collezione di **uno o più pacchetti**.
- Unità di **Gestione delle Dipendenze** e Controllo di Versione.
- Definito dal file `go.mod`.
- Importazione: Usa il percorso completo (`nome-modulo/pacchetto`).
- Comando: `go mod init nome-modulo` crea la radice del progetto.



UTILIZZARE I PACCHETTI

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    // Stampa un numero casuale tra 0 e 9
    fmt.Println("Il mio numero preferito è", rand.Intn(10))
}
```



LIBRERIA STANDARD VS. ESTERNA

- Go fornisce una **libreria standard** con pacchetti comuni:
<https://pkg.go.dev/std>.
- Puoi creare pacchetti anche **all'interno del tuo progetto**.
- Puoi usare pacchetti esterni o crearne di nuovi da condividere.



CREARE PACCHETTI ALL'INTERNO DEL TUO PROGETTO



STRUTTURA DEL PROGETTO

1. All'interno della directory del tuo progetto, `go mod init nome-modulo` crea un modulo (`go.mod` e `go.sum`).
2. A partire da quel momento, puoi creare **sotto-moduli** (sotto-directory) per i pacchetti.
3. Li importi usando il percorso completo: `nome-modulo/sotto-dir`.




```
main.go  
go.mod  
go.sum  
package1/  
    functions.go  
    other-things.go  
subpackage/  
    file.go
```



ESEMPIO: PACCHETTO INTERNO

Contenuto di `package1/functions.go`:

```
package package1

// Nota: Dobbiamo usare la lettera MAIUSCOLA come prima
// lettera per rendere questa funzione pubblica.
// Altrimenti sarà disponibile solo all'interno di package1.
func Dummy() {}

// Questa funzione è privata (non esportata)
func internalFunction() {}
```



Contenuto di main.go:

```
package main

import "nome-modulo/package1" // Importa il pacchetto interno

func main() {
    // Chiama la funzione pubblica Dummy
    package1.Dummy()
    // package1.internalFunction() // Errore: non è esportata
}
```



MODULI/PACCHETTI ESTERNI



OTTENERE DIPENDENZE ESTERNE

Per utilizzare moduli esterni, devi prima "ottenerli" (scaricarli).

Comando:

```
$ go get gopkg.in/yaml.v2
```

Questo comando scarica il pacchetto `gopkg.in/yaml.v2` e lo aggiunge a `go.mod`



UTILIZZO DEL PACCHETTO ESTERNO

Esempio di importazione e utilizzo:

```
package main

import "gopkg.in/yaml.v2" // Pacchetto esterno

func main() {
    data := map[string]string{
        "name": "Go",
        "type": "Language",
    }

    // Marshalling (conversione in YAML)
    yamlData, _ := yaml.Marshal(data)
    // fmt.Println(string(yamlData))
}
```



GO TOOLCHAIN



CREARE E DEFINIRE IL MODULO

Il comando `go mod` è fondamentale per inizializzare e gestire il progetto.

Comando	Descrizione
<code>go mod init <path/nome></code>	Inizializza un nuovo modulo Go. Crea il file <code>go.mod</code> nella directory corrente, definendo il percorso radice del modulo (es. <code>github.com/utente/repo</code>).
<code>go mod tidy</code>	Pulisce e Aggiorna. Aggiunge eventuali dipendenze mancanti (trovate nel codice) e rimuove quelle inutilizzate dal file <code>go.mod</code> e <code>go.sum</code> .
☐ <code>go mod download</code>	Scarica tutti i moduli richiesti in <code>go.mod</code> nella cache locale.

STRUTTURA STANDARD (LAYOUT)

I progetti Go seguono una convenzione semplice:

- **File Sorgente:** I file Go finiscono con `.go`.
- **Package `main`:** Contiene la funzione `main()` ed è il punto d'ingresso per un programma eseguibile.
- **Package (Nome Personalizzato):** Contiene librerie e logica di business riutilizzabile.



Esempio:

1. Crea la cartella: `mkdir myproject && cd myproject`
2. Inizializza il modulo: `go mod init example.com/myproject`
3. Crea il file principale: `touch main.go`



RUN ED ESECUZIONE RAPIDA

Per eseguire rapidamente il codice senza produrre un eseguibile finale.

Comando	Descrizione
<code>go run main.go</code>	Compila ed esegue il codice sorgente specificato. Utile per testare rapidamente durante lo sviluppo.
<code>go run .</code>	Compila ed esegue il package <code>main</code> nella directory corrente.



COMPILAZIONE E BUILD

Per creare un eseguibile binario distribuibile.

Comando	Descrizione
<code>go build .</code>	Compila il package corrente (o main se eseguibile) e genera un file binario nella directory corrente, chiamato come il modulo o la directory.
<code>go build -o app_name .</code>	Compila e specifica il nome del file di output (es. app_name).
<code>go install .</code>	Compila e installa l'eseguibile nella cartella binari predefinita di Go (\$GOPATH/bin o \$GOBIN).



TEST

Il framework di testing è integrato nella toolchain.

Comando	Descrizione
<code>go test</code>	Esegue i test (file con suffisso <code>_test.go</code>) nella directory corrente.
<code>go test ./...</code>	Esegue i test in tutte le sottodirectory.
<code>go test -v</code>	Esegue i test in modalità verbosa (mostra i dettagli di ogni test).



FORMATTAZIONE DEL CODICE

Go impone uno stile di formattazione standardizzato.

Comando	Descrizione
<code>go fmt</code> •	Riformatta automaticamente tutti i file Go nella directory corrente in modo standardizzato, utilizzando lo stile canonico di Go.



ANALISI E LINTING (CONTROLLO QUALITÀ)

Strumenti integrati per l'analisi statica del codice.

Comando	Descrizione
<code>go vet .</code>	Analizza il codice sorgente per identificare potenziali errori (bug) o costrutti sospetti, come errori comuni di formattazione di stringhe o assegnazioni inutili.
<code>go doc <package/func></code>	Mostra la documentazione per un pacchetto o una funzione specifici. (es. <code>go doc fmt.Println</code>).



ALTRI STRUMENTI UTILI

Comandi per la gestione e l'ispezione dell'ambiente.

Comando	Descrizione
<code>go get <path/package></code>	Aggiunge un nuovo pacchetto esterno al file <code>go.mod</code> e lo scarica.
<code>go version</code>	Mostra la versione di Go installata.
<code>go env</code>	Stampa le variabili d'ambiente di Go (es. <code>\$GOPATH</code> , <code>\$GOBIN</code>).



AGGIUNTA E AGGIORNAMENTO

I comandi gestiscono le dipendenze esterne richieste dal codice.

Comando	Descrizione
<code>go get <path/package></code>	Aggiunge una nuova dipendenza al progetto (o aggiorna una esistente) e la registra in <code>go.mod</code> .
<code>go get <path/package>@v1.2.3</code>	Scarica e utilizza una specifica versione o un tag.
<code>go get -u ./...</code>	Aggiorna tutte le dipendenze del modulo alla versione patch o minor più recente.
☐ <code>go get -u=patch ./...</code>	Aggiorna solo alle versioni patch più recenti (più sicuro).

ISPEZIONE E BLOCCO (GO.SUM)

Il file `go.sum` contiene gli hash crittografici per le dipendenze, garantendo l'integrità.

Comando	Descrizione
<code>go list -m all</code>	Elenca tutte le dipendenze del modulo, incluse quelle transitive (di cui dipendono le tue dipendenze).
<code>go mod vendor</code>	Crea una cartella <code>vendor/</code> contenente le copie locali di tutte le dipendenze, per build isolate o per reti limitate.
<code>go mod verify</code>	Verifica che i moduli scaricati nella cache Go corrispondano agli hash in <code>go.sum</code> .



PULIZIA

Per mantenere pulito l'ambiente del modulo.

Comando	Descrizione
<code>go clean -modcache</code>	Cancella la cache dei moduli (\$GOPATH/pkg/mod), forzando il download di tutte le dipendenze alla prossima build/run.



